

TFRL-MD: Transformer-enhanced Federated Reinforcement Learning for Intelligent Microservice Deployment and Resource Allocation

Keli Liu, Jing Yang*, *Member, IEEE*, Xiaoli Ruan, Yuling Chen, Xu Wang, Jianhong Cheng, Yulong Yang and Minyi Guo, *Fellow, IEEE*

Abstract—Edge computing provides low-latency execution environments for delay-sensitive microservices by deploying computational resources at the network periphery. However, microservice deployment and resource allocation in geographically distributed, heterogeneous edge environments must simultaneously navigate challenges such as data privacy preservation, heterogeneous resource constraints, and dynamic workload fluctuations. Therefore, we propose TFRL-MD, a Transformer-enhanced federated reinforcement learning framework for privacy-preserving and intelligent microservice deployment and resource allocation. The framework models each edge node as an independent Actor-Critic agent to learn deployment policies via local training, enabling cross-node collaborative optimization without sharing raw environmental data. To improve the quality of knowledge fusion among heterogeneous edge nodes, we design a Transformer-based Embedding-Space Aggregation Mechanism (EASM) that leverages multi-head self-attention to adaptively fuse their intermediate-layer knowledge representations. Furthermore, a dual-channel hybrid objective training mechanism is constructed to synchronously inject the aggregated global representations into the optimization targets of both Actor and Critic networks as differentiable semantic embeddings, thereby smoothly guiding local gradient descent and eliminating the policy oscillations inherent in traditional parameter overwriting. Dynamic simulation experiments, conducted using the EUA dataset from the Australian Central Business District (CBD) region, demonstrate that TFRL-MD outperforms various baselines in both average response time and load balancing, achieving maximum improvements of 46.01% and 61.11%, respectively.

Index Terms—Cloud services, microservices, resource allocation, federated learning, reinforcement learning

I. INTRODUCTION

This work was supported by the national natural science foundation of China (62562013), Scientific and Technological Innovation Platform Research Project of Guizhou Province (CXPTXM[2025]024), the science and technology project of Guizhou Province (QKHZC[2025]023, QKHZC[2025]022, QKHZC[2025]025), Science and Technology Research Project of Guizhou Provincial Department of Education(No.QJJ[2025]014). (Corresponding author: Jing Yang.)

Keli Liu, Jing Yang, Xiaoli Ruan, Yuling Chen and Minyi Guo are with the State Key Laboratory of Public Big Data, Guizhou University, Guiyang, 550025, China (e-mail: gs.kliu23@gzu.edu.cn; jyang23@gzu.edu.cn; xlruan@gzu.edu.cn; ylchen3@gzu.edu.cn; myguo@gzu.edu.cn).

Xu Wang is with the Key Laboratory of Electronic Composites of Guizhou Province, College of Big Data and Information Engineering, Guizhou University, Guiyang, Guizhou, 550025 China (e-mail:xuwang@gzu.edu.cn)

Jianhong Cheng and Yulong Yang are with Institute of Guizhou Aerospace Measuring and Testing Technology, Guiyang 550025, China (e-mail:jianhong_cheng@csu.edu.cn, gzu_ylyang@163.com).

WITH the rapid proliferation of Internet of Things (IoT) devices and the continuous advancement of Artificial Intelligence (AI) [1], latency-sensitive applications such as intelligent surveillance [2] and real-time speech recognition [3] have emerged, imposing higher requirements on low-latency response and high-reliability service deployment. Traditional centralized cloud computing [4], [5] often introduces significant transmission latency across Wide Area Networks (WANs) due to the geographical distance from end-users, making it difficult to satisfy the stringent real-time demands of emerging applications. To this end, edge computing [6], [7] provides low-latency, distributed execution support for complex applications by deploying computational resources at the network periphery.

In edge computing architectures, complex applications are typically decomposed into microservices with functional dependencies and are deployed across geographically distributed, heterogeneous edge nodes for collaborative execution [8], [9], [10]. However, microservice deployment and resource allocation is not a simple mapping problem; rather, it is a sophisticated optimization challenge [11] influenced by resource heterogeneity, workload dynamics, and privacy constraints. Capabilities across edge nodes vary significantly, while user requests fluctuate over time and space. Furthermore, cross-regional collaborative decision-making involves sensitive information regarding user distributions and resource states. Therefore, how to achieve efficient microservice deployment in dynamic, heterogeneous environments while preserving privacy has become a critical research problem in the field of edge computing.

Existing microservice deployment methods are mainly categorized into centralized and distributed approaches. Early methods primarily adopt heuristic algorithms and mathematical programming [12], [13], which achieve promising solving efficiency in static or weakly dynamic scenarios. However, they heavily rely on prior knowledge and struggle to adapt to continuously changing dynamic environments. In contrast, Deep Reinforcement Learning (DRL) demonstrates stronger adaptability in dynamic microservice deployment problems by virtue of its ability to continuously interact with the environment and autonomously learn decision-making policies [14], [15]. Nevertheless, centralized DRL schemes face non-negligible information leakage risks when handling cross-region privacy-sensitive data. Multi-Agent Deep Reinforcement Learning (MADRL) alleviates this bottleneck by dis-

tributing decision-making authority to individual edge nodes. However, most MADRL methods still rely on sharing local observations or global states among agents, which also poses privacy exposure risks in cross-region deployment scenarios and causes the communication overhead to grow super-linearly as the node scale increases.

Federated Reinforcement Learning (FRL) combines the privacy-preserving mechanisms of federated learning with the dynamic decision-making capabilities of reinforcement learning, providing a new avenue for achieving collaborative optimization under privacy constraints [16], [17]. However, existing FRL methods still exhibit distinct limitations at both the knowledge fusion and training mechanism levels. On the one hand, mainstream methods perform a weighted average of model parameters from each node during the global aggregation phase. In heterogeneous edge environments, this operation tends to dilute the specialized decision-making knowledge accumulated by individual nodes in specific deployment scenarios, thereby restricting the representational capacity of the global model. On the other hand, existing methods [18], [19] typically decouple the global aggregation process from the local training loop, where the aggregated results are propagated back to local models through parameter replacement only after a communication round ends. This makes it difficult for cross-node collaborative knowledge to provide real-time guidance for local gradient updates, easily inducing deployment policy oscillations.

To address the aforementioned challenges, we propose TFRL-MD, a Transformer-enhanced Federated Reinforcement Learning framework for privacy-preserving and intelligent Microservice Deployment and resource allocation in edge computing environments. Based on Federated Twin Delayed Deep Deterministic Policy Gradient (TD3), this framework models each edge node as an independent Actor-Critic agent that learns deployment policies via local training without uploading raw environmental data. To improve the quality of knowledge fusion among heterogeneous nodes, we design an Embedding-Space Aggregation Mechanism (EASM) based on Transformer. EASM extends the object of federated collaboration from the model parameter space to the semantic embedding space, leveraging multi-head self-attention to perform adaptive weighted fusion of the intermediate-layer knowledge representations from each node. Subsequently, we further design a dual-channel global knowledge injection training mechanism. This mechanism synchronously introduces the aggregated global embeddings into both the Critic value evaluation and Actor policy optimization processes, enabling cross-node collaborative knowledge to participate in local gradient updates in real time during each training iteration. Our main contributions are summarized as follows:

- A privacy-preserving distributed microservice deployment framework based on Federated TD3, namely TFRL-MD, is proposed. It models each edge node as an independent Actor-Critic agent that learns deployment policies exclusively through local environmental interactions, inherently reducing the risk of sensitive information exposure.
- A Transformer-based Embedding-Space Aggregation

Mechanism (EASM). By extending federated collaboration from the traditional parameter space to the semantic embedding space, EASM dynamically fuses the intermediate-layer knowledge representations of various nodes via multi-head self-attention, enhancing the global model's ability to capture heterogeneous deployment experiences.

- A dual-channel global knowledge injection training mechanism. To eliminate the collaborative experience latency caused by decoupled global aggregation and local optimization in conventional FRL, this paper constructs a symmetric dual-channel hybrid objective function that synchronously injects global knowledge into Actor and Critic networks as differentiable semantic embeddings, effectively suppressing policy oscillations induced by traditional parameter overwriting.
- We construct a 24-hour continuous dynamic simulation environment based on the real-world EUA dataset from the Australian CBD region, and design four types of cross-region scenarios covering different scales and topological structures for comprehensive evaluation. Compared to existing baselines, TFRL-MD improves the average response time by up to 46.01% and enhances load balancing by 61.11%.

The remainder of this article is organized as follows. Section II analyzes related work. Section III describes the system model and problem definition. Section IV elaborates on the design of the TFRL-MD. Section V provides a comprehensive performance evaluation and comparative analysis. Finally, Section VI concludes the article and discusses future research directions.

II. RELATED WORK

A. Centralized Microservice Deployment Methods

Extensive research has been conducted to optimize microservice deployment strategies in edge computing. Early studies primarily relied on heuristic algorithms and mathematical programming for deployment decision-making [12], [13], [20], [21], [22]. For instance, Lee et al. [20] proposed the Dependency-Aware Task Allocation (DATA) algorithm, which assigns tasks to various edge nodes by perceiving inter-microservice dependencies to reduce application completion time. However, such methods often depend on stringent system prior knowledge and lack real-time adaptability to dynamic environmental changes, making it difficult to navigate complex edge scenarios where user requests and resource states fluctuate continuously.

To overcome these limitations, researchers have introduced DRL to address the challenges posed by the dynamics of microservice deployment [14], [15], [23], [24], [25], [26], [27], [28]. For example, Fu et al. [29] modeled the resource management problem as a Markov Decision Process (MDP) and employed a Dueling Deep Q-Network (DQN) for its solution. Nevertheless, their aggressive resource partitioning strategy led to low resource utilization while preventing Quality of Service (QoS) violations. Regarding edge server resource scheduling, Lu et al. [28] constructed a multidimensional

resource-aware decision model that jointly optimizes the allocation of CPU, memory, and I/O resources using Advantage Actor-Critic (A2C) DRL. Yet, their reward function design neglected the statistical dispersion of resource distributions, resulting in spatial resource imbalances that struggle to cope with bursty workloads and local hotspots. Recently, fusion frameworks combining Graph Neural Networks (GNNs) and RL have gained significant attention [30], [31], [32]. Chen et al. [32] captured features of edge resource distributions and microservice invocation relationships through multi-channel directed graph convolutional networks and graph attention networks, integrated with sequence-to-sequence models for multi-step decision optimization.

However, the aforementioned centralized methods face the following limitations: (i) existing static or single-agent deployment policies struggle to accurately capture state evolution patterns over specific time intervals, which severely restricts real-time perception and self-adaptive adjustment capabilities; (ii) traditional centralized approaches incur significant information leakage risks when processing privacy-sensitive data in geographically distributed edge environments, failing to satisfy the security requirements of practical microservice deployment scenarios.

B. Multi-Agent Collaborative Deployment and Resource Scheduling

To overcome the information bottlenecks inherent in centralized architectures, MADRL [33] delegates decision-making authority to individual edge nodes. By employing the Centralized Training and Decentralized Execution (CTDE) paradigm, MADRL achieves local independent inference and global policy coordination, effectively satisfying the low-latency requirements of dynamic edge environments.

MADRL has been extensively applied to the fields of edge task offloading and resource allocation. Du et al. [34] proposed a joint service placement and task offloading optimization method based on Multi-Agent Deep Deterministic Policy Gradient (MADDPG). By modeling UAV-based edge servers as multiple agents, they unified the optimization of service instance deployment, offloading decisions, admission control, and resource allocation, mapping continuous action outputs to mixed-integer discrete decisions. In multi-user multi-server edge computing systems, Lu et al. [35] formulated the task partitioning and resource allocation problem as a Partially Observable Markov Game (POMG) for multi-base station servers, utilizing the CTDE-based MADDPG framework for its solution to reduce task failure rates while enhancing system utility. Li et al. [36] introduced the Cooperative Multi-agent Deep Reinforcement Learning (CMDRL) algorithm, which employs dual agents to optimize cost and revenue objectives separately, achieving distributed computation offloading for heterogeneous tasks; however, their approach neglects the dynamic coordination issues arising from node mobility. For Internet of Vehicles (IoV) edge computing, Zhang et al. [37] proposed a Mobility-aware Coordinated Multi-agent (MCMA) reinforcement learning method that integrates trajectory prediction. By leveraging an Informer network to predict vehicle

movement to guide task migration, they utilized a two-stage framework combining Multi-Agent Proximal Policy Optimization (MAPPO) and MADDPG to jointly optimize offloading decisions and resource allocation, significantly reducing task completion latency.

However, most existing MADRL approaches rely on agents sharing global state information or directly exchanging local observations. In cross-regional deployment scenarios, this inevitably exposes privacy-sensitive data, such as user distributions and resource workloads. Furthermore, as the number of edge nodes increases, the communication overhead between nodes grows superlinearly, making it difficult to sustain large-scale distributed deployment scenarios.

C. Federated Reinforcement Learning for Privacy-Preserving Edge Optimization

To balance privacy preservation with collaborative optimization, FRL integrates the privacy-protecting mechanisms of FL with the dynamic decision-making capabilities of RL. FRL has emerged as a significant research direction in edge resource management.

Xiao et al. [16] applied Federated Deep Q-Networks (FDQN) to edge task offloading decisions, modeling user equipments as distributed agents to minimize overall energy consumption while satisfying task latency constraints. Jia et al. [38] investigated the joint optimization of resource allocation and data selection in a federated edge learning environment, designing a low-complexity algorithm by combining the Convex-Concave Procedure (CCP) and gradient projection methods. However, the CCP approach only guarantees convergence to a local optimum, often trapping the system in sub-optimal states during intensive resource competition. Tong et al. [39] designed a federated DQN offloading framework for Directed Acyclic Graph (DAG) task dependency scenarios and introduced an automated hyperparameter optimization mechanism; nevertheless, this approach still relies on model parameter-weighted fusion, making it difficult to capture the heterogeneous deployment expertise across various nodes. Chen et al. [18] proposed PFR-OA, a personalized federated reinforcement learning framework for multi-edge smart communities, which incorporates a proximal term based on TD3 to stabilize training and designs a partial-greedy participant selection mechanism to lower aggregation complexity. However, its synchronization mechanism based on discrete communication rounds fails to align global knowledge with local gradient updates at a training-step level, resulting in a significant latency in collaborative experience dissemination.

Despite these efforts, existing FRL methods face several critical limitations. First, they primarily rely on weighted averaging in the parameter space, which tends to dilute specialized decision-making knowledge accumulated by nodes in specific deployment scenarios and leads to convergence bias under heterogeneous data distributions. Second, existing methods decouple global aggregation from local training, relying on coarse-grained parameter replacement that both delays cross-node knowledge dissemination and risks destabilizing local policy optimization.

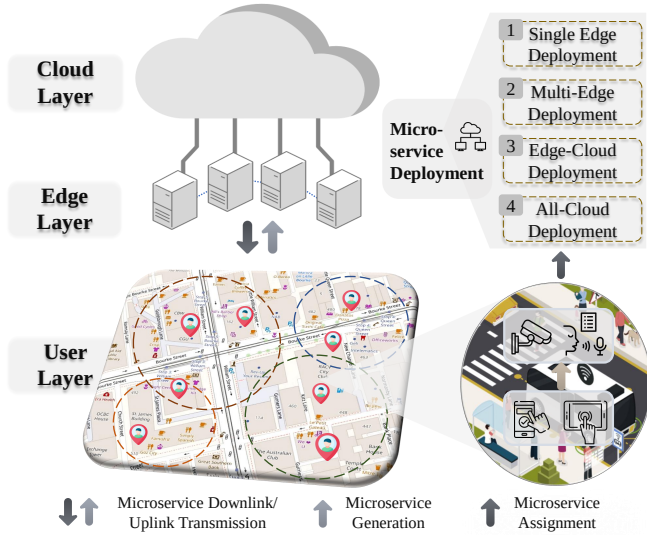


Fig. 1. Microservice Deployment Architecture (The Cloud Layer and Edge Layer contain distributed nodes supporting four deployment strategies, while the User Layer illustrates: (I) geographically distributed IoT devices generating dynamically varying service request loads, and (II) dynamic microservice generation and allocation processes based on user demands.)

III. SYSTEM MODEL AND PROBLEM STATEMENT

The system architecture, computation model and problem formulation are defined in this section.

A. System Architecture

As illustrated in Fig. 1, we construct a three-tier heterogeneous edge computing architecture, comprising a cloud data center C (Cloud Layer), a set of edge servers $E = \{e_1, e_2, \dots, e_N\}$ (Edge Layer), and a set of mobile users $U = \{u_1, u_2, \dots, u_M\}$ (User Layer). The main notations are summarized in Table I.

The cloud layer consists of a central cloud data center with abundant computing resources. Each edge node e_i is uniquely characterized by its geographical location (x_i^e, y_i^e) , service radius R_i and computing resource capacity (C_i^e, M_i^e) . Edge nodes are interconnected via high-speed links with bandwidth denoted by b_{ij} , forming a service network that covers different geographical regions. User locations (x_k^u, y_k^u) and generated service request volumes $Q(t)$ dynamically vary over time within a 24-hour period $T_t \in \{0, 1, \dots, 23\}$.

Each user request targets a specific application $a_n \in \{a_1, a_2, \dots, a_k\}$, where different application categories have distinct microservice composition requirements. Each application comprises a service chain formed by a set of microservices $\{s_1, s_2, \dots, s_n\}$ with strict sequential dependencies, where the computational output of microservice s_i serves as the input for s_{i+1} . Each microservice $s_i \in S$ has specific resource requirements (C_i^s, M_i^s) and is deployed on an edge or cloud server through a resource allocation mechanism. The connection between user terminals and edge nodes is constrained by the coverage radius $d(u, i) < R_i$. When user u initiates a service request at time t , the set of available edge nodes is first determined. If available nodes exist, the optimal node is selected to provide the service; otherwise, the request is

TABLE I
NOTATION DESCRIPTION

Notation	Meaning
C	Central cloud data center
E	Set of heterogeneous edge servers, $E = \{e_1, e_2, \dots, e_N\}$
$U(t)$	Set of active mobile users at time t
a_n	Application instance
s_i	Microservice instance within a service chain
C_i^s, M_i^s	CPU and Memory resource requirements of microservice s_i
$T_{tot}^u(t)$	The total response time of user's request at time t
T_{avg}	Average end-to-end response time across all requests
$\rho_{cpu}^j, \rho_{mem}^j$	CPU and Memory utilization rates of edge node e_j
$\sigma_{cpu}^2, \sigma_{mem}^2$	CPU and Memory load variances across edge nodes
L	Comprehensive cluster load balancing degree
$\omega_{cpu}, \omega_{mem}$	Resource weighting factors where $\omega_{cpu} + \omega_{mem} = 1$
s_t, a_t	Local environment state and continuous deployment action
$e_{\pi_i}^{(t)}, e_{Q_i}^{(t)}$	The actor and critic network embedding vectors of edge node i
$z_{\pi}^{agg(t)}, z_Q^{agg(t)}$	Global aggregated embeddings
λ	Knowledge fusion balance factor

forwarded to the cloud server for processing. Optimal node selection comprehensively considers both distance and node resource status.

We introduce the following definitions for the service execution phase:

Definition 1 (Uplink Transmission Time): The time required for a user device to transmit initial request data. Let $UT_{u,j}^t$ and $UT_{u,c}^t$ denote the uplink transmission time from user u to edge node e_j or cloud server C at time t , respectively. Here, $UT_{u,j}^t = D_{req}/B_{u,j}^t$, where D_{req} is the request data volume and $B_{u,j}^t$ is the bandwidth.

Definition 2 (Execution Completion Time): The time required for microservice s_m to complete all computational tasks at a node. Let $ET_{m,j}^t$ denote the execution completion time of microservice s_m at node e_j at time t , where $ET_{m,j}^t = ST_{m,j}^t + (W_m)/C_j^t$. Here, $ST_{m,j}^t$ is the start execution time, W_m is the microservice computational load, and C_j^t is the computing capability of node e_j . When microservices are executed in series across multiple nodes, the start execution time of a subsequent microservice depends on the completion time of the preceding microservice and the data transmission time between nodes, i.e., $ST_{m+1,j'}^t = ET_{m,j}^t + TT_{j,j'}^t$, where $TT_{j,j'}^t$ is the data transmission time between node e_j and $e_{j'}$.

Definition 3 (Downlink Transmission Time): The time required for result data to be transmitted back to the user device from the final execution node after service processing is complete. Let $DT_{j,u}^t$ denote the downlink transmission time from node e_j to user u at time t .

B. Computation Model

1) Latency Model: Based on the previously defined service execution phases, we analyze the end-to-end response time for different microservice deployment scenarios. According to the microservice deployment location, the latency model can be classified into the following four cases:

Case 1: Single Edge Deployment. When all microservices $\{s_1, s_2, \dots, s_n\}$ of an application are deployed on the same

edge node e_j , the total response time is:

$$T_{tot}^e(t) = UT_{u,j}^t + \sum_{i=1}^n ET_{i,j}^t + DT_{j,u}^t \quad (1)$$

Case 2: Multi-Edge Deployment. When the microservices of an application are distributed across multiple edge nodes, the total response time includes inter-node data transmission time:

$$T_{tot}^{mul}(t) = UT_{u,j_1}^t + \sum_{i=1}^n ET_{i,j_i}^t + \sum_{i=1}^{n-1} TT_{j_i,j_{i+1}}^t + DT_{j_n,u}^t \quad (2)$$

where j_i represents the edge node for which microservice s_i is deployed, and $TT_{j_i,j_{i+1}}^t = D_{i,i+1}/b_{j_i,j_{i+1}}$ denotes the data transmission time between edge nodes.

Case 3: Edge-Cloud Collaborative Deployment. When edge server resources are insufficient and some microservices $\{s_k, s_{k+1}, \dots, s_n\}$ need to be deployed in the cloud, the total response time is:

$$T_{tot}^{hy}(t) = UT_{u,j_1}^t + \sum_{i=1}^{k-1} ET_{i,j_i}^t + \sum_{i=1}^{k-2} TT_{j_i,j_{i+1}}^t + TT_{j_{k-1},c}^t + \sum_{i=k}^n ET_{i,c}^t + DT_{c,u}^t \quad (3)$$

where $TT_{j_{k-1},c}^t$ represents the data transmission time from the last edge node to the cloud, $ET_{i,c}^t$ represents the execution time of the microservice in the cloud, and $DT_{c,u}^t$ represents the downlink transmission time from the cloud to the user.

Case 4: All-Cloud Deployment. When edge resources are completely insufficient or the user is beyond the coverage range of all edge nodes, all microservices will be deployed in the cloud, and the total response time is:

$$T_{tot}^c(t) = UT_{u,c}^t + \sum_{i=1}^n ET_{i,c}^t + DT_{c,u}^t \quad (4)$$

where $UT_{u,c}$ and $DT_{c,u}$ represent the uplink and downlink transmission times from the user to the cloud, respectively.

Considering all user requests, the average response time T_{avg} is defined as:

$$T_{avg} = \frac{1}{24} \sum_{t=1}^{24} \frac{\sum_{u \in U(t)} T_{tot}^u}{|U(t)|} \quad (5)$$

where T_{tot}^u represents the total response time of user's request at time t , and $|U(t)|$ represents the number of active users at time t .

2) Resource Consumption Model: Resource consumption primarily includes CPU resources and Memory resources. The available resources of an edge node e_j are defined as:

$$R_{avail}^e = [C_{avail}^e, M_{avail}^e] \quad (6)$$

where C_{avail}^e and M_{avail}^e respectively represent the available CPU and Memory resources of edge node e_j . When microservice s_k is deployed on edge node e_j , the resource consumption is updated as:

$$C_{avail}^e = C_{avail}^e - C_{req}^{s_k} \quad (7)$$

$$M_{avail}^e = M_{avail}^e - M_{req}^{s_k} \quad (8)$$

where $C_{req}^{s_k}$ and $M_{req}^{s_k}$ respectively represent the CPU and Memory resources required by microservice s_k .

3) Load Balancing Model: We adopt a variance-based load balancing evaluation method, comprehensively considering the balance of CPU and Memory resources. For an edge node $e_j \in E$, its CPU and Memory utilization rates are defined as:

$$\rho_{cpu}^j = 1 - \frac{C_{avail}^e}{\max C_{tot}^k} \quad (9)$$

$$\rho_{mem}^j = 1 - \frac{M_{avail}^e}{\max M_{tot}^k} \quad (10)$$

where $\max C_{tot}^k$ and $\max M_{tot}^k$ respectively represent the maximum CPU and Memory capacities of edge nodes. The resource utilization vector is represented as $\rho^j = [\rho_{cpu}^j, \rho_{mem}^j]^T$.

To achieve efficient resource utilization and avoid node overload, we construct a variance-based load balancing evaluation model. The CPU and Memory load variances for all edge nodes are defined as:

$$\sigma_{cpu}^2 = \frac{1}{|E|} \sum_{j=1}^{|E|} (\rho_{cpu}^j - \bar{\rho}_{cpu})^2 \quad (11)$$

$$\sigma_{mem}^2 = \frac{1}{|E|} \sum_{j=1}^{|E|} (\rho_{mem}^j - \bar{\rho}_{mem})^2 \quad (12)$$

where $\bar{\rho}_{cpu}$ and $\bar{\rho}_{mem}$ are the average CPU and memory utilization rates of all nodes, respectively. The load variances are normalized to obtain $\tilde{\sigma}_{cpu}$ and $\tilde{\sigma}_{mem}$. The comprehensive load balancing degree L is defined as the weighted sum of CPU and Memory load variances:

$$L = \omega_{cpu} \cdot \tilde{\sigma}_{cpu}^2 + \omega_{mem} \cdot \tilde{\sigma}_{mem}^2 \quad (13)$$

where ω_{cpu} and ω_{mem} are weighting factors, and $\omega_{cpu} + \omega_{mem} = 1$.

C. Problem Formulation

Based on the aforementioned computational model, we formalize the microservice deployment problem in an edge-cloud environment as a bi-objective optimization problem, aiming to simultaneously optimize average response time and load balancing degree. This problem can be stated as:

$$\min_D \{T_{avg}(D), L(D)\} \quad (14)$$

where $D = \{D_{i,j} \mid 1 \leq i \leq N_S, 1 \leq j \leq N_E\}$ represents the microservice deployment decision, $D_{i,j} \in \{0, 1\}$ indicates the deployment status of microservice s_i on edge node e_j . $T_{avg}(D)$ and $L(D)$ represent the system average response time and load balancing degree, respectively, given by Eq. (5) and Eq. (13).

IV. THE PROPOSED TFRL-MD

This section introduces TFRL-MD, a Transformer-enhanced Federated Reinforcement Learning framework, to address the challenges of microservice deployment and resource allocation under privacy preservation constraints.

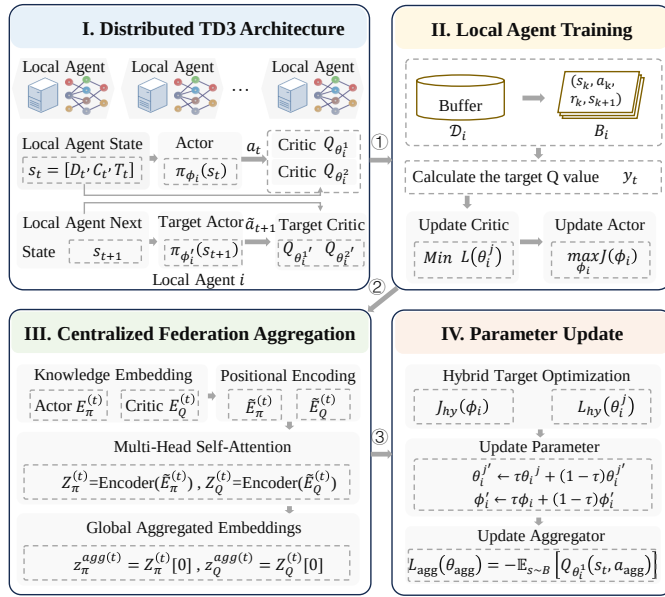


Fig. 2. TFRL-MD Framework (where I. represents distributed TD3 agents across edge nodes with independent Actor-Critic networks, II. shows local agent training using buffered experiences and TD3 optimization, III. employs Transformer-based federated aggregation with multi-head self-attention for knowledge fusion, and IV. implements hybrid parameter updates for federated learning integration).

A. Overview of TFRL-MD

The overall architecture of TFRL-MD is illustrated in Fig. 2. In the distributed TD3 architecture (Process I), each edge node maintains an independent Actor-Critic network and learns deployment policies solely from local environmental interactions, without uploading raw data. In the local agent training phase (Process II), each agent samples from its private replay buffer and updates its Critic and Actor networks according to the standard TD3 objective. Periodically, the federated aggregation process (Process III) collects intermediate-layer embeddings from all agents, fuses them via a Transformer encoder [40], and extracts a global aggregated embedding through the CLS token. Finally, in the parameter update phase (Process IV), the aggregated global embeddings are injected back into each agent's local optimization through hybrid objective functions, enabling cross-node collaborative knowledge to participate in both Critic value estimation and Actor policy optimization within each training iteration. The deployment objective is to minimize user request response time and achieve balanced computational resource utilization.

1) *Algorithm Procedure*: The proposed TFRL-MD process is illustrated in Algorithm 1. The algorithm first initializes the Actor-Critic networks and Transformer aggregator parameters for each agent (Line 1). During each training iteration, agents select and execute actions from the current state s_t , obtaining rewards and next state information (Lines 5-6), followed by storing transition samples into the experience replay buffer (Line 7). Throughout the learning process, the algorithm first samples batch data from the replay buffer and computes target Q-values (Lines 10-11), then alternately updates the Critic networks, Actor networks, and Transformer aggregator param-

Algorithm 1: TFRL-MD Framework

- 1: **Initialize**: Actor-Critic networks $(\pi_{\phi_i}, Q_{\theta_i^j})$, target networks $(\pi_{\phi_i'}, Q_{\theta_i^{j'}})$, Transformer aggregator parameters θ_{agg} and experience replay buffers $\mathcal{D}_i$.
- 2: **for** episode = 1 to E_{max} **do**
- 3: **for** time step $t = 1$ to T_{max} **do**
- 4: **for** each agent $i \in N$ **do**
- 5: Observe state s_t^i and select action $a_t^i = \pi_{\phi_i}(s_t^i) + \epsilon$.
- 6: Execute action a_t^i and observe reward r_t^i and next state s_{t+1}^i .
- 7: Store transition $(s_t^i, a_t^i, r_t^i, s_{t+1}^i)$ in replay buffer $\mathcal{D}_i$.
- 8: **Local Training**:
- 9: **for** each agent $i \in N$ **do**
- 10: Sample batch B_i from replay buffer $\mathcal{D}_i$.
- 11: Compute target Q-value y_t using Eq. (17).
- 12: Update Critic networks by minimizing $L(\theta_i^j)$ using Eq. (26).
- 13: **if** $t \bmod d = 0$ **then**
- 14: Update Actor network by maximizing $J(\phi_i)$ using Eq. (27).
- 15: Soft update target networks using Eq. (28) and Eq. (29).
- 16: **Federated Aggregation**:
- 17: **if** $t \bmod T_{fed} = 0$ **then**
- 18: Extract embeddings $e_{\pi_i}^{(t)}$ and $e_{Q_i}^{(t)}$ using Eq. (18) and Eq. (19).
- 19: Form embedding sets $E_{\pi}^{(t)}$ and $E_Q^{(t)}$ using Eq. (20) and Eq. (21).
- 20: Add CLS tokens and positional encoding using Eq. (22) and Eq. (23).
- 21: Compute global aggregated embeddings using Eq. (25).
- 22: Update local networks with hybrid objectives using Eq. (30) and Eq. (31).
- 23: Update Transformer aggregator using Eq. (32).
- 24: **return** Optimized deployment policy π^*

eters until convergence (Lines 12-15). In the federated aggregation phase, global knowledge fusion is performed every T_{fed} steps by extracting embedding vectors $e_{\pi_i}^{(t)}$ and $e_{Q_i}^{(t)}$ from each node (Line 18), constructing aggregated embedding sets (Lines 19-20), utilizing the Transformer encoder to generate global aggregated embeddings (Line 21), and ultimately updating local network parameters through hybrid objective functions (Lines 22-23).

2) *Computational Complexity*: The computational complexity of TFRL-MD can be comprehensively evaluated through analysis of key processes. The algorithm's network parameter initialization complexity is $O(P)$, where P represents the total number of neural network parameters. In each training iteration, the action selection complexity is $O(d_s \cdot d_h)$, where d_s denotes the state dimension and d_h represents the hidden layer dimension. The complexity of sampling batch data from the replay buffer is $O(B)$, where B is the batch size. The most computationally intensive component involves updating the dual Critic network parameters θ_i^j and Actor network parameters ϕ_i through gradient descent, with each operation exhibiting complexity $O(B \cdot P)$. In multi-agent scenarios, the Transformer aggregation process demonstrates complexity $O(B \cdot N \cdot d_h \cdot L)$, where N represents the number of agents and L denotes the number of Transformer layers. The soft update operation for target networks exhibits complexity $O(P)$. Consequently, the overall computational complexity of TFRL-MD is $O(T \times N \times B \times P + \frac{T^2}{T_{fed}} \times N^2 \times d_h^2)$, where T represents the total time steps and T_{fed} denotes the federated aggregation frequency.

B. Actor-Critic Architecture

We model the microservice deployment problem as an MDP consisting of multiple edge nodes. The local environment of each node is defined as a five-tuple $M = (S, A, P, R, \gamma)$, where:

- 1) **State Space** S . At time t , the system state $s_t \in S$ is defined as $s_t = [D_t, C_t, T_t]$, encompassing the microservice deployment matrix $D_t = \{D_{i,j}^t\}$. The resource state matrix C_t represents the CPU and Memory utilization of each edge node, and the time identifier $T_t \in \{0, 1, \dots, 23\}$ represents a 24-hour time period, with each segment corresponding to a different user request distribution.
- 2) **Action Space** A . The action $a_t \in A$ represents the microservice deployment decision. To address the exponential explosion of the action space when directly representing microservice deployment decisions, we design a continuous action space $a_t \in \mathbb{R}^{(N_s \times N_e)}$, where $a_t[i, j]$ represents the probability of deploying microservice s_i to an edge node e_j . The action values are constrained to the range $[-1, 1]$ by the Actor network's tanh activation function, and are subsequently mapped to discrete deployment decisions, ensuring that each microservice is deployed on exactly one node.
- 3) **Transition Function** P . $P(s_{t+1}|s_t, a_t)$ describes the system state evolution dynamics after executing action a_t , primarily influenced by changes in microservice deployment state and resource state. The deployment state is deterministically updated based on the current action $D_{t+1} = f_D(D_t, a_t)$, while the resource state is affected by the randomness of user requests $C_{t+1} = f_C(D_{t+1}, C_t, \omega_t)$, where ω_t represents environmental stochasticity (such as sudden user requests).
- 4) **Reward Function** R . The immediate reward $R(s_t, a_t)$ is obtained upon executing action a_t in state s_t , guiding the agent towards optimization goals. We design a multi-objective reward function that comprehensively considers the response time optimization reward R_T and the load optimization reward R_L , which is defined as $R(s_t, a_t) = w_T R_T(s_t, a_t) + w_L R_L(s_t, a_t)$.

We employ a TD3-based Actor-Critic architecture [41]. The core function of the Actor network is policy generation, mapping environmental states to optimal service deployment actions. For each agent $i \in N$, the Actor network $\pi_{\phi_i} : S \rightarrow A$ maps states to deployment actions through a two-layer MLP with layer normalization:

$$\pi_{\phi_i}(s_t) = \tanh(f_{\pi}^2(\text{LN}(f_{\pi}^1(s_t)))) \quad (15)$$

where $f_{\pi}^j(x) = \text{ReLU}(W_{\pi,j} \cdot x + b_{\pi,j})$. The Critic network is responsible for value evaluation, computing the cumulative expected return of state-action pairs. TD3 employs a double Critic to mitigate the overestimation issue of Q-values. The dual Critic networks $Q_{\theta_i^j} : S \times A \rightarrow \mathbb{R}, j \in \{1, 2\}$ adopt a similar architecture but receive state-action pairs as input:

$$Q_{\theta_i^j}(s_t, a_t) = f_Q^3(f_Q^2(\text{LN}(f_Q^1([s_t, a_t]))) \quad (16)$$

where f_Q^k represents the transformation function of the k -th layer. When computing the target Q-value, the dual Q-

networks select the smaller value estimated by the two networks:

$$y_t = r_t + \gamma \min_{j=1,2} Q_{\theta_i^j}(s_{t+1}, \tilde{a}_{t+1}) \quad (17)$$

where $\tilde{a}_{t+1} = \pi_{\phi_i}(s_{t+1}) + \epsilon$ is the target action with added noise, and $\gamma = 0.99$. To enable knowledge sharing in the federated stage, an embedding extraction mechanism is applied to the intermediate layers of both networks, producing knowledge vectors $h_{\pi_i}(s_t)$ and $h_{Q_i^j}(s_t, a_t)$ that serve as the input to EASM.

C. Transformer-based Federated Aggregation

In microservice deployment scenarios, the user request patterns, resource constraints, and workload characteristics across different edge nodes often exhibit significant disparities. Simple parameter averaging tends to dilute the localized, specialized knowledge acquired by individual nodes within their specific deployment environments, thereby weakening the representational capacity of the global model. To this end, we propose the Transformer-based Embedding-Space Aggregation Mechanism (EASM), which extends federated collaboration from the parameter space to the semantic embedding space.

In contrast to the direct aggregation of model parameters, EASM first extracts intermediate layer feature representations from the Actor and Critic networks of each agent, serving as a semantically compressed representation of the current batch of input states and decision-making processes. These intermediate embeddings reflect a node's deployment expertise and decision-making preferences in its current environment more directly than raw parameters, making them a more suitable medium for cross-node knowledge fusion. The embedding representation for each agent is a weighted feature representation extracted from a batch of samples $B_i = (s_k, a_k, r_k, s_{k+1})$, adaptively weighted through an attention mechanism to capture the implicit knowledge of local deployment decisions:

$$e_{\pi_i}^{(t)} = \sum_{k=1}^{|B_i|} \alpha_{ik}^{(t)} \cdot h_{\pi_i}(s_k) \quad (18)$$

$$e_{Q_i}^{(t)} = \sum_{k=1}^{|B_i|} \alpha_{ik}^{(t)} \cdot \frac{1}{2} [h_{Q_i^1}(s_k, a_k) + h_{Q_i^2}(s_k, a_k)] \quad (19)$$

where $h_{\pi_i}(s_k)$ and $h_{Q_i^j}(s_k, a_k)$ denote the feature representations extracted from the intermediate layers of the Actor and Critic networks, respectively. $\alpha_{ik}^{(t)} = \frac{\exp(f_{att}(h_i(s_k)))}{\sum_{j=1}^{|B_i|} \exp(f_{att}(h_i(s_j)))}$ is the attention weight. We collect the actor network embedding vectors $e_{\pi_i}^{(t)}$ and critic network embedding vectors $e_{Q_i}^{(t)}$ from each agent to form the aggregated embedding sets:

$$E_{\pi}^{(t)} = \{e_{\pi_1}^{(t)}, e_{\pi_2}^{(t)}, \dots, e_{\pi_N}^{(t)}\} \quad (20)$$

$$E_Q^{(t)} = \{e_{Q_1}^{(t)}, e_{Q_2}^{(t)}, \dots, e_{Q_N}^{(t)}\} \quad (21)$$

To aggregate global sequential information and differentiate the relative importance of various agents, a CLS token and positional encoding are introduced:

$$\tilde{E}_{\pi}^{(t)} = [x_{cls}^{\pi}; E_{\pi}^{(t)}] + P_{pos} \quad (22)$$

$$\tilde{E}_Q^{(t)} = [x_{cls}^Q; E_Q^{(t)}] + P_{pos} \quad (23)$$

where x_{cls} is a learnable CLS token specifically designed to capture the most relevant knowledge representation for the current microservice deployment decision. P_{pos} is the positional encoding, used to capture the relative positional information of agents within the sequence.

Through the multi-head self-attention mechanism of the Transformer encoder, the deployment experiences of different edge nodes are dynamically weighted based on their relevance. The Multi-Head Self-Attention mechanism captures contextual information in different representation subspaces through parallel attention computations:

$$\text{FA}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (24)$$

The encoder processing is as follows:

$$Z_\pi^{(t)} = \text{Encoder}(\tilde{E}_\pi^{(t)}), \quad Z_Q^{(t)} = \text{Encoder}(\tilde{E}_Q^{(t)}) \quad (25)$$

Finally, the output at the CLS position is extracted as the global aggregated embeddings $z_\pi^{\text{agg}(t)} = Z_\pi^{(t)}$ and $z_Q^{\text{agg}(t)} = Z_Q^{(t)}$. These vectors encode the collective deployment knowledge distilled from all agents and are passed to the federated training process.

D. Federated Training Process

The federated training process in TFRL-MD is designed to integrate global knowledge directly into the local optimization loop. Rather than operating on a decoupled schedule, the framework injects the aggregated global embeddings into each agent's ongoing Actor and Critic objectives immediately following an aggregation event. This allows cross-node collaborative experience to actively guide the local gradient updates across the subsequent training steps, ensuring continuous knowledge dissemination.

In each training iteration, agent i randomly samples a batch of experiences B_i from its experience replay buffer D_i , capturing deployment experiences from different periods. The optimization objective for the Critic network is to minimize the mean squared error between the predicted Q-value and the target Q-value:

$$L(\theta_i^j) = E[(Q_{\theta_i^j}(s_t, a_t) - y_t)^2] \quad (26)$$

where the target Q-value y_t is calculated by Eq. (17). TFRL-MD employs a delayed policy update mechanism, updating the Actor network only after multiple updates of the Critic network. The optimization objective for the Actor network is to maximize the Q-value estimation:

$$\max_{\phi_i} J(\phi_i) = E_{s \sim D}[Q_{\theta_i^1}(s_t, \pi_{\phi_i}(s_t))] \quad (27)$$

To enhance training stability, a soft update mechanism is utilized to maintain the target network parameters:

$$\theta_i^{j'} \leftarrow \tau \theta_i^j + (1 - \tau) \theta_i^{j'} \quad (28)$$

$$\phi_i' \leftarrow \tau \phi_i + (1 - \tau) \phi_i' \quad (29)$$

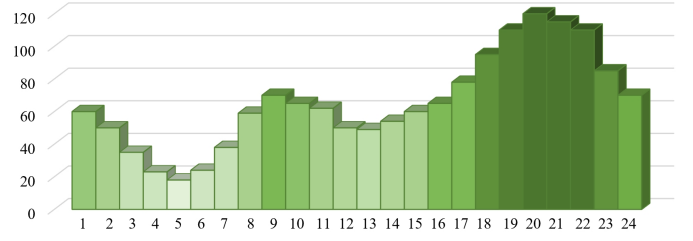


Fig. 3. Request Traffic Across 24-Hour Time Slots

Every T_{fed} local steps, the global aggregated embeddings $z_\pi^{\text{agg}(t)}$ and $z_Q^{\text{agg}(t)}$ are injected through hybrid objective functions that blend local and global learning signals:

$$J_{hy}(\phi_i) = \lambda J(\phi_i) + (1 - \lambda) f_\pi(e_{\pi_i}^{(t)}, z_\pi^{\text{agg}(t)}) \quad (30)$$

$$L_{hy}(\theta_i^j) = \lambda L(\theta_i^j) + (1 - \lambda) f_Q(e_{Q_i}^{(t)}, z_Q^{\text{agg}(t)}) \quad (31)$$

where $\lambda \in [0, 1]$ denotes the knowledge fusion balance factor, which dynamically weights the trade-off between local policy optimization and global semantic knowledge alignment. f_π and f_Q are learned projection functions that map the aggregated embeddings into the action space and value space, respectively, bridging the gap between the global semantic representation and node-specific deployment decisions. The Transformer aggregator updates its parameters via the Adam optimizer, with the learning objective of maximizing the action value generated based on aggregated knowledge:

$$L_{agg}(\theta_{agg}) = -E_{s \sim B}[Q_{\theta_{agg}^1}(s_t, a_{agg})] \quad (32)$$

where $a_{agg} = \pi_{out}(f_{agg}(E; \theta_{agg}))$ denotes the action generated based on aggregated knowledge. By maximizing the action value generated from aggregated knowledge, distributed decision knowledge is effectively integrated.

V. PERFORMANCE EVALUATION

This section evaluates the proposed TFRL-MD and compares it with benchmark methods.

A. Experimental Setup

1) *Simulation Settings*: The algorithm's performance is evaluated within a 24-hour continuous simulation environment, constructed using the authentic EUA dataset [42] sourced from the Australian Central Business District (CBD).

• *Resource Configuration*. Four heterogeneous edge nodes are deployed with service radii of [150, 70, 130, 80] meters, and inter-node network bandwidth ranging from 100-400 Mbps, accurately simulating the coverage capabilities of various types of base stations and edge servers. Network transmission parameters are set as follows: user-to-edge node at 1000 Mbps, user-to-cloud at 10 Mbps, and edge-to-cloud at 500 Mbps, corresponding to transmission delays of 500ms and 1000ms, respectively. The central cloud is configured with infinite computing resources, undertaking microservice deployment when edge resources are insufficient.

- **Applications and Microservices.** We designed 20 applications of varying complexity, with complexity tiered by the number of microservices. The system includes 15 heterogeneous microservice types, with resource requirements ranging from 2-15 MB for CPU and 2-26 GB for Memory. Microservice allocation adheres to the application's business logic and involves clear dependencies and data flow constraints.

- **User Behavior.** The experiment adopts a 24-hour load variation pattern based on real user behavior (Fig. 3), where the request volume gradually increases from a nighttime low of 18 requests to an afternoon peak of 120 requests, reflecting a typical urban user's daily activity pattern. 47 users are randomly deployed and dynamically move within each time window, simulating realistic mobility scenarios. User requests follow an 80-20 distribution principle, meaning 80% of requests are concentrated on 4 popular applications, while 20% are randomly distributed among the other 16 applications. Users support concurrent multi-application requests, and request arrival times follow a Poisson process.

- **Four Cross-Regional Scenarios.** Scenario 1 (Baseline) simulates a standard edge network architecture along a traffic artery, consisting of 4 servers and 47 terminal users. Scenario 2 comprises 5 servers and 60 users, where specific hotspots are introduced to intensify the polarization of local workloads. Scenario 3 consists of 6 servers and 70 users, simulating the spatial mismatch between uniform infrastructure coverage and non-uniform user distribution. Scenario 4 scales to a large-scale CBD scenario involving 8 servers and 100 users.

2) **Benchmark Methods:** To comprehensively evaluate the performance of the proposed method, we select the following baselines:

- **Twin Delayed Deep Deterministic Policy Gradient (TD3)** [41]: The original TD3 algorithm without federated learning.
- **Proximal Policy Optimization (PPO)** [43]: A policy gradient-based reinforcement learning algorithm that constrains policy update magnitude through a clipped objective function to ensure training stability.
- **FedSAC** [44]: It combines federated learning with the Soft Actor-Critic (SAC) algorithm, generating a global model by aggregating local parameters from each agent.
- **DDPG** [45]: A classic continuous control reinforcement learning algorithm that integrates DQN's experience replay mechanism with policy gradient methods to directly output continuous actions.
- **Greedy** [46]: A traditional baseline method that makes resource allocation decisions based on a locally optimal strategy.

3) **Model Configuration:** We implement TFRL-MD using the PyTorch deep learning framework, leveraging GPU parallel computing to accelerate the training process. TFRL-MD adopts a federated learning architecture, configured with 4 independent agents for collaborative learning, each maintaining an independent Actor-Critic network structure. The network architecture employs a three-layer fully connected design and integrates a Transformer module with multi-head attention mechanisms. Agents operate in a synchronous federated learning mode, with model aggregation performed every 10 training steps. The aggregation strategy utilizes a Transformer-based

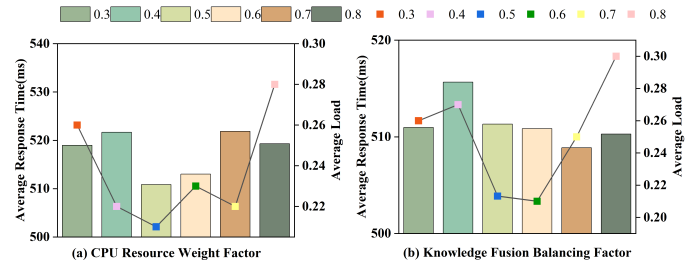


Fig. 4. Sensitivity Analysis of Parameters.

attention weight allocation mechanism to facilitate knowledge sharing among agents. The experiment is set for 100 training episodes, with each episode comprising 24 continuous time segments simulating a full daily cycle. All experiments are trained and evaluated in parallel on a server equipped with an Intel i9-12900K CPU and an NVIDIA RTX 4090 GPU.

B. Results Analysis

We evaluate the proposed TFRL-MD framework using two primary performance metrics: Average Response Time and Average Load Balancing, as defined by Eq. (5) and (13), respectively. Our comprehensive experimental evaluation examines TFRL-MD's deployment performance across multiple dimensions, including varying user request traffic loads, microservice resource demands, application quantities, and resource capacity constraints.

1) **Parameter Sensitivity Analysis:** To investigate the impact of key hyperparameters on the deployment performance of TFRL-MD, we conduct an in-depth sensitivity analysis on the CPU resource weight factor (ω_{cpu} , see Eq. (13)) and the knowledge fusion balance factor (λ , see Eq. (30) and Eq. (31)).

Fig. 4(a) illustrates the impact of varying ω_{cpu} on the algorithm's performance. When $\omega_{cpu} = 0.5$, TFRL-MD achieves the optimal average response time (approximately 510 ms) and the lowest average load (about 0.21), which corresponds to the equal-weight configuration of CPU and memory resources. Deviating from this equilibrium point leads to a conspicuous degradation in deployment performance; although the variation in response time remains minor, the load-balancing degree noticeably deteriorates. Therefore, $\omega_{cpu} = 0.5$ is uniformly adopted in subsequent experiments.

Fig. 4(b) displays the sensitivity of the system to the knowledge fusion balance factor λ . The results indicate that the algorithm performance peaks when $\lambda = 0.6$. This demonstrates that such a configuration provides the most sufficient cross-node collaborative guidance for the agents' decision-making. Notably, when the factor exceeds 0.7, the over-reliance on global aggregated embeddings suppresses the adaptability of local policies to node-specific environmental dynamics, causing the average load to surge to 0.30 when $\lambda = 0.8$. Consequently, we set the default value of λ to 0.6.

2) **Impact of Request Traffic on Performance:** To evaluate the performance of all six schemes under different request traffic loads, we set the request traffic to 50% of its default value (100%) and then gradually increased it to 200% for comparative analysis. As shown in Fig. 5, with increasing

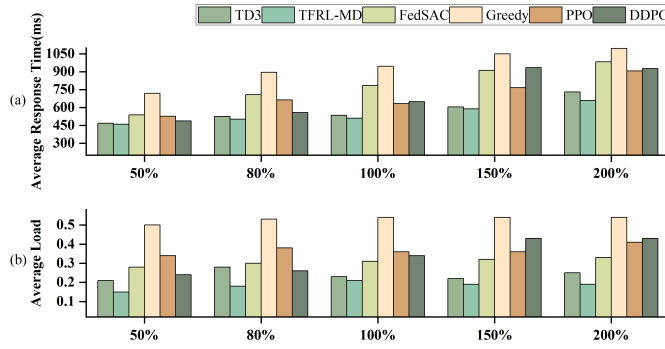


Fig. 5. Performance Comparison Under Dynamic Request Traffic

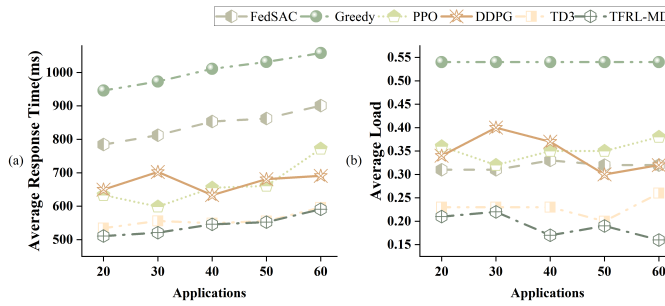


Fig. 6. Performance Comparison Under Application Numbers

request traffic, the average response time of all schemes shows an upward trend, but with significant differences in growth rates. TFRL-MD and TD3 exhibit the strongest load adaptability, with relatively smooth increases in response time. In contrast, FedSAC and Greedy are extremely sensitive to traffic changes, with response times sharply deteriorating in the 150%-200% high-load range, where Greedy's peak even exceeds 1000ms. This is primarily due to increased traffic leading to intensified server resource competition, which in turn causes microservice response delays to grow. Notably, TFRL-MD significantly outperforms TD3 in terms of system load, indicating that the federated learning mechanism achieves more efficient resource utilization through inter-device collaboration, effectively avoiding resource allocation imbalance.

3) *Impact of Application Numbers on Performance:* We conducted a comprehensive evaluation of the performance of all six schemes by gradually increasing the number of applications from 20 to 60. As shown in Fig. 6, when server resources remain fixed, an increase in the number of application requests inevitably leads to intensified resource contention, which in turn significantly increases the average response time of tasks. In contrast, TFRL-MD consistently maintains optimal deployment performance across all test ranges, with response times stably remaining within the 500-600ms range and the smallest increase. Generally, most deployment methods compromise load balancing performance when pursuing low response time optimization. For example, although TD3 performs suboptimally in terms of response time, its average load value reaches 0.20-0.25. However, TFRL-MD maintains

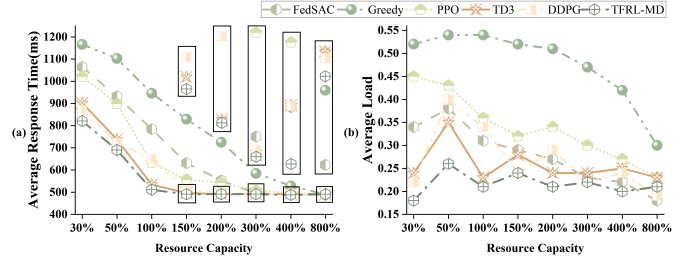


Fig. 7. Performance Comparison Under Resource Capacity Settings

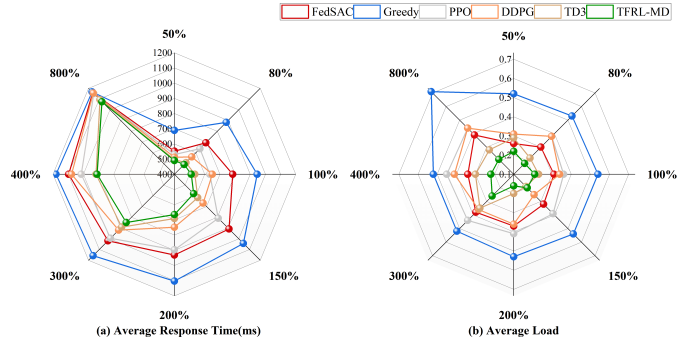


Fig. 8. Performance Comparison Under Different Resource Requirements

its average load value at a low level of 0.15-0.20 while simultaneously achieving optimal response time. This advantage is attributed to TFRL-MD's improved policy network structure and optimized reward function design, enabling it to better balance the multi-objective optimization requirements of microservice deployment, thereby effectively addressing the load pressure brought by an increasing number of applications.

4) *Performance Comparison Under Varying Resource Capacities:* We evaluated the performance of all six schemes under various resource capacity settings, adjusting the resources of each server from 30% to 800% of their default values. As shown in Fig. 7, the deployment performance gap among all methods gradually narrows as resource capacity increases, eventually tending to converge within the 400% to 800% range. This is because sufficient server resources significantly reduce resource contention pressure among microservices, thereby diminishing the impact of inter-server collaborative optimization on overall deployment performance. Among them, Greedy shows the most significant improvement in response time; however, its performance regarding system load remains suboptimal, reflecting a critical issue where the scheme achieves performance at the expense of excessive resource consumption, indicating a lack of resource utilization efficiency. In contrast, reinforcement learning-based schemes demonstrate superior optimization capabilities across all resource levels. Notably, under the extreme constraint of 30% resource capacity, TFRL-MD significantly outperforms other methods. This demonstrates its ability to achieve more precise utilization of fragmented resources, maintaining efficient deployment through intelligent cross-node collaboration even under severely limited resource conditions.

5) *Impact of Resource Requirements on Performance:* To evaluate the impact of different microservice resource

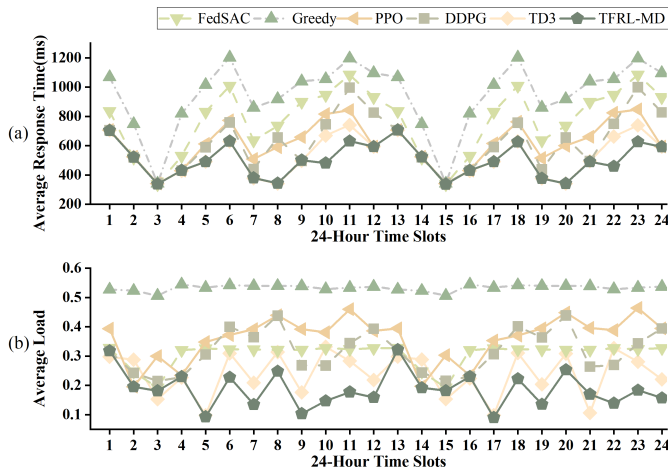


Fig. 9. Performance Comparison Across 24-Hour Time Slots

requirements on deployment performance, we extended the microservice resource requirements from the default 50% to an extreme of 800%. As illustrated in Fig. 8(a), the response time performance of all methods degrades as resource requirements increase. Within the high-load interval of 400% to 800%, the average response times of traditional reinforcement learning methods (such as PPO and DDPG) soar rapidly and approach 1100ms, indicating these methods reach the threshold of system processing capacity when handling heavy workloads. In contrast, TFRL-MD exhibits the most stable and gradual increase in response time. As shown in Fig. 8(b), the average server load generally rises with the increase in resource requirements. Notably, the load of the Greedy approaches 0.6 after the demand reaches 300%, while TFRL-MD consistently maintains a level around 0.2. This advantage is attributed to its Transformer-enhanced federated aggregation mechanism, which maximizes edge server resource utilization, and its multi-node collaborative mechanism, which achieves intelligent load balancing, effectively alleviating deployment pressure caused by increased resource demand. The experimental results fully confirm that efficient resource allocation strategies and cross-node collaborative optimization are key factors influencing microservice deployment performance.

6) *Comparison with Baselines Over 24 Hours*: To validate the performance of various algorithms under different request load conditions, we compared the deployment efficiency of the six methods over a 24-hour cycle. As can be seen from Fig. 9, all methods exhibit clear diurnal periodicity, with performance troughs occurring between 2:00-4:00 and 13:00-16:00, and peaking during morning and evening hours. Among them, TFRL-MD performs optimally, with deployment response times consistently maintained within the 300-700ms range, demonstrating good performance even during peak periods; conversely, the Greedy algorithm performs the worst, showing the highest deployment latency and server load. This is primarily due to the greedy strategy's reliance on fixed decision rules, which tend to select the most powerful servers, leading to imbalanced resource allocation and localized overload. In contrast, TFRL-MD achieves global state awareness through

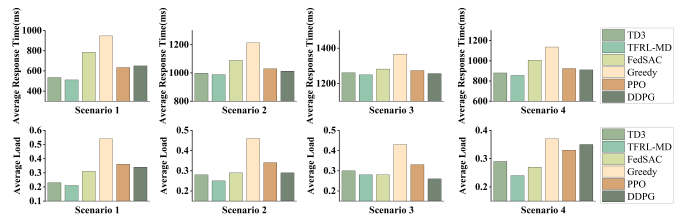


Fig. 10. Performance Comparison Under Four Cross-Regional Scenarios

its federated learning mechanism, enabling devices to make collaborative optimal decisions, effectively avoiding the local suboptimal issues of TD3, thereby demonstrating significant performance advantages in complex dynamic environments.

7) *Performance Comparison Under Four Cross-Regional Scenarios*: To evaluate the generalization of TFRL-MD across diverse geographical topologies and network scales, we conducted comparative experiments in four distinct cross-regional scenarios. As illustrated in Fig. 10, TFRL-MD consistently exhibits the most optimal and robust deployment performance across all scenarios. In Scenarios 1 and 2, owing to the EASM mechanism's ability to perceive local hotspots, the average response time remains stable between 500ms and 1000ms, significantly outperforming Greedy and FedSAC. Even in the largest-scale Scenario 4, where the numbers of nodes and users double, TFRL-MD successfully maintains the response latency at approximately 850ms. From the perspective of server load, TFRL-MD consistently maintains the lowest average load levels, ranging from 0.20 to 0.28. In contrast, TD3 suffers from significantly aggravated load imbalance in Scenario 4 due to its lack of adaptive perception of the global environment. These experimental results demonstrate that TFRL-MD possesses superior resource scheduling efficiency when dealing with non-uniform user distributions and large-scale concurrent requests.

VI. CONCLUSION

To address the critical challenges of privacy preservation, resource heterogeneity, and dynamic workload fluctuations in microservice deployment and resource allocation within edge computing environments, we propose TFRL-MD, a Transformer-enhanced Federated Reinforcement Learning framework. Based on Federated Twin Delayed Deep Deterministic Policy Gradient (TD3), this approach models each edge node as an independent Actor-Critic agent, enabling privacy-preserving and distributed collaborative decision-making through local training. To enhance the quality of knowledge fusion across heterogeneous nodes, we further design the Embedding-Space Aggregation Mechanism (EASM) based on Transformers and employ a dual-channel global knowledge injection strategy to strengthen the guidance of aggregated knowledge on local policy optimization and value estimation. Experimental results demonstrate that TFRL-MD outperforms existing baselines, achieving a maximum improvement of 46.01% in average response time and an enhancement of 61.11% in load balancing metrics. In future work, we will focus on integrating Dynamic Graph Neural Networks to explicitly model the spatio-temporal evolution

of time-varying network topologies and heterogeneous edge resources, thereby further enhancing the adaptive capabilities of microservice orchestration.

REFERENCES

- [1] W. Cao, X. Tang, T. Deng, R. Cao, and K. Li, "Resource-aware dynamic scheduling for tasks with deadline constraints on edge computing systems," *IEEE Transactions on Cloud Computing*, vol. 13, no. 4, pp. 1119–1133, 2025.
- [2] A. Aryan, D. P. Singh, and N. Kumar, "Intelligent video surveillance system," in *2024 International Conference on Expert Clouds and Applications (ICOECA)*, 2024, pp. 971–976.
- [3] M. N. Kakade and D. B. Salunke, "Real-time speech recognition using convolutional neural network," in *Advanced Machine Intelligence and Signal Processing*. Springer, 2022, pp. 467–477.
- [4] K. Peng, Y. Hu, H. Ding, H. Chen, L. Wang, C. Cai, and M. Hu, "Large-scale service mesh orchestration with probabilistic routing in cloud data centers," *IEEE Transactions on Services Computing*, 2025.
- [5] T. Bhardwaj, H. Upadhyay, and S. C. Sharma, "Autonomic resource provisioning framework for service-based cloud applications: a queueing-model based approach," in *2020 10th international conference on cloud computing, data science & engineering (confluence)*. IEEE, 2020, pp. 605–610.
- [6] A. M. Maia and Y. Ghamri-Doudane, "A deep reinforcement learning approach for the placement of scalable microservices in the edge-to-cloud continuum," in *GLOBECOM 2023-2023 IEEE Global Communications Conference*. IEEE, 2023, pp. 479–485.
- [7] C. Sun, X. Li, C. Wang, Q. He, X. Wang, and V. C. Leung, "Hierarchical deep reinforcement learning for joint service caching and computation offloading in mobile edge-cloud computing," *IEEE Transactions on Services Computing*, vol. 17, no. 4, pp. 1548–1564, 2024.
- [8] K. Liu, J. Yang, X. Ruan, H. Zhao, S. Sun, S. Li, and M. Guo, "Samd-srg: Service-aware microservice deployment using spectral ranking and graph-enhanced reinforcement learning," *IEEE Transactions on Cognitive Communications and Networking*, vol. 12, pp. 1649–1663, 2026.
- [9] K. Peng, H. Wen, Z. Guo, H. Ge, C. Cai, B. Zhou, and M. Hu, "Energy-efficient joint deployment and routing for delay-sensitive microservices in edge computing," *IEEE Transactions on Sustainable Computing*, vol. 11, no. 1, pp. 1–14, 2026.
- [10] X. He, Z. Tu, M. Wagner, X. Xu, and Z. Wang, "Online deployment algorithms for microservice systems with complex dependencies," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1746–1763, 2023.
- [11] X. Zhou, J. Yang, Y. Li, S. Li, Z. Su, and J. Lu, "Ec-trl: Evolutionary-weighted clustering and transformer-augmented reinforcement learning for dynamic resource scheduling in edge cloud environments," *IEEE Internet of Things Journal*, vol. 12, no. 6, pp. 7503–7517, 2025.
- [12] Y. Hu, H. Wang, L. Wang, M. Hu, K. Peng, and B. Veeravalli, "Joint deployment and request routing for microservice call graphs in data centers," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 11, pp. 2994–3011, 2023.
- [13] S. Wang, Z. Ding, and C. Jiang, "Elastic scheduling for microservice applications in clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 1, pp. 98–115, 2020.
- [14] K. Peng, J. He, J. Guo, Y. Liu, J. He, W. Liu, and M. Hu, "Delay-aware optimization of fine-grained microservice deployment and routing in edge via reinforcement learning," *IEEE Transactions on Network Science and Engineering*, 2024.
- [15] X. Tian, H. Meng, Y. Shen, J. Zhang, Y. Chen, and Y. Li, "Dynamic microservice deployment and offloading for things-edge-cloud computing," *IEEE Internet of Things Journal*, 2024.
- [16] H. Xiao, Z. Hu, X. Zhang, A. Xu, M. Zheng, and K. Li, "Federated deep reinforcement learning for task offloading in mec-enabled heterogeneous networks," *IEEE Internet of Things Journal*, 2024.
- [17] S. S. Shinde, A. Bozorgchenani, D. Tarchi, and Q. Ni, "On the design of federated learning in latency and energy constrained computation offloading operations in vehicular edge computing systems," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 2, pp. 2041–2057, 2021.
- [18] Z. Chen, B. Xiong, X. Chen, G. Min, and J. Li, "Joint computation offloading and resource allocation in multi-edge smart communities with personalized federated deep reinforcement learning," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 11 604–11 619, 2024.
- [19] S. Khodadadian, P. Sharma, G. Joshi, and S. T. Maguluri, "Federated reinforcement learning: Linear speedup under markovian sampling," in *International conference on machine learning*. PMLR, 2022, pp. 10 997–11 057.
- [20] J. Lee, H. Ko, J. Kim, and S. Pack, "Data: Dependency-aware task allocation scheme in distributed edge clouds," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 12, pp. 7782–7790, 2020.
- [21] L. Gu, Z. Chen, H. Xu, D. Zeng, B. Li, and H. Jin, "Layer-aware collaborative microservice deployment toward maximal edge throughput," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 71–79.
- [22] M. Abd Elaziz, I. Attiya, L. Abualigah, M. Iqbal, A. Ali, A. Al-Fuqaha, and S. El-Sappagh, "Hybrid enhanced optimization-based intelligent task scheduling for sustainable edge computing," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 889–898, 2023.
- [23] F. Song, H. Xing, X. Wang, S. Luo, P. Dai, and K. Li, "Offloading dependent tasks in multi-access edge computing: A multi-objective reinforcement learning approach," *Future Generation Computer Systems*, vol. 128, pp. 333–348, 2022.
- [24] X. Zhou, J. Yang, Y. Li, S. Li, and Z. Su, "Deep reinforcement learning-based resource scheduling for energy optimization and load balancing in sdn-driven edge computing," *Computer Communications*, vol. 226, p. 107925, 2024.
- [25] K. Gasmi, K. Abassi, L. Romdhani, and O. Debauche, "Reinforcement learning-based approach for microservices-based application placement in edge environment," in *2023 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2023, pp. 1098–1103.
- [26] K. Liu, J. Yang, X. Ruan, H. Zhao, S. Sun, S. Li, and M. Guo, "Samd-srg: Service-aware microservice deployment using spectral ranking and graph-enhanced reinforcement learning," *IEEE Transactions on Cognitive Communications and Networking*, vol. 12, pp. 1649–1663, 2026.
- [27] X. Zhou, J. Yang, Y. Li, S. Li, Z. Su, and J. Lu, "Ec-trl: Evolutionary-weighted clustering and transformer-augmented reinforcement learning for dynamic resource scheduling in edge cloud environments," *IEEE Internet of Things Journal*, 2024.
- [28] J. Lu, J. Yang, S. Li, Y. Li, W. Jiang, J. Dai, and J. Hu, "A2c-drl: Dynamic scheduling for stochastic edge-cloud environments using a2c and deep reinforcement learning," *IEEE Internet of Things Journal*, 2024.
- [29] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo, "Adaptive resource efficient microservice deployment in cloud-edge continuum," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1825–1840, 2021.
- [30] L. Chen, Y. Bai, P. Zhou, Y. Li, Z. Qu, and J. Xu, "On adaptive edge microservice placement: A reinforcement learning approach endowed with graph comprehension," *IEEE Transactions on Mobile Computing*, 2024.
- [31] C. Wang, H. Yu, X. Li, F. Ma, X. Wang, T. Taleb, and V. C. Leung, "Dependency-aware microservice deployment for edge computing: A deep reinforcement learning approach with network representation," *IEEE Transactions on Mobile Computing*, 2024.
- [32] S. Chen, Q. Yuan, J. Li, H. He, S. Li, X. Jiang, and J. Yang, "Graph neural network aided deep reinforcement learning for microservice deployment in cooperative edge computing," *IEEE Transactions on Services Computing*, 2024.
- [33] B. Ai, G. Ye, Z. Wu, and Y. Sun, "Overlapping coalition formation-enabled noncooperative game-combined multi-agent drl for uav-assisted resource allocation," *IEEE Transactions on Sustainable Computing*, vol. 11, no. 1, pp. 29–41, 2026.
- [34] J. Du, Z. Kong, A. Sun, J. Kang, D. Niyato, X. Chu, and F. R. Yu, "Maddpg-based joint service placement and task offloading in mec empowered air-ground integrated networks," *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 10 600–10 615, 2024.
- [35] K. Lu, R.-D. Li, M.-C. Li, and G.-R. Xu, "Maddpg-based joint optimization of task partitioning and computation resource allocation in mobile edge computing," *Neural Comput. Appl.*, vol. 35, no. 22, p. 16559–16576, Apr. 2023. [Online]. Available: <https://doi.org/10.1007/s00521-023-08527-8>
- [36] K. Li, X. Wang, Q. He, B. Yi, A. Morichetta, and M. Huang, "Cooperative multiagent deep reinforcement learning for computation offloading: A mobile network operator perspective," *IEEE Internet of Things Journal*, vol. 9, no. 23, pp. 24 161–24 173, 2022.
- [37] X. Zhang, C. Wang, Y. Zhu, J. Cao, and T. Liu, "Multi-agent deep reinforcement learning with trajectory prediction for task migration-assisted computation offloading," *IEEE Transactions on Mobile Computing*, vol. 24, no. 7, pp. 5839–5856, 2025.

- [38] Y. Jia, Z. Huang, J. Yan, Y. Zhang, K. Luo, and W. Wen, "Joint optimization of resource allocation and data selection for fast and cost-efficient federated edge learning," *IEEE Transactions on Cognitive Communications and Networking*, 2024.
- [39] Z. Tong, J. Deng, J. Mei, Y. Zhang, and K. Li, "Multi-objective dag task offloading in mec environment based on federated dqn with automated hyperparameter optimization," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3999–4012, 2024.
- [40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [41] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*. PMLR, 2018, pp. 1587–1596.
- [42] G. Cui, Q. He, F. Chen, H. Jin, Y. Xiang, and Y. Yang, "Location privacy protection via delocalization in 5g mobile edge computing environment," *IEEE Transactions on Services Computing*, vol. 16, no. 1, pp. 412–423, 2021.
- [43] W. Zhao, C. Wu, R. Zhong, K. Shi, and X. Xu, "Edge computing and caching optimization based on ppo for task offloading in rsu-assisted iov," in *2023 IEEE 9th World Forum on Internet of Things (WF-IoT)*. IEEE, 2023, pp. 01–06.
- [44] J. Qian, Y. Jiang, X. Liu, Q. Wang, T. Wang, Y. Shi, and W. Chen, "Federated reinforcement learning for electric vehicles charging control on distribution networks," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 5511–5525, 2023.
- [45] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.
- [46] K. Cheng, S. Zhang, C. Tu, X. Shi, Z. Yin, S. Lu, Y. Liang, and Q. Gu, "Proscale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1294–1312, 2023.



Xiaoli Ruan received the PhD degree in information and communication engineering from Yunnan University, in 2021. Participating in the "Jiaxiu Zhiguang" Visiting Scholar Program at Zhejiang University from September 2024 to June 2025. She is currently a lecturer with the State Key Laboratory of Public Big Data, Guizhou University, China. Her research interests include Artificial Intelligence and Multimodal Big Data Fusion. She has published more than 20 peer-reviewed papers in the related area, including well-archived international journals such as IEEE Transactions on Geoscience and Remote Sensing, Knowledge-Based Systems, and IEEE Internet of Things Journal.



Yuling Chen received her BS from Taishan University, Taian, PR China, in 2006; MS from Guizhou University, Guiyang, PR China, in 2009; Ph.D. from Guizhou University, Guiyang, PR China, in 2021. She is now a professor at the State Key Laboratory of Public Big Data, Guizhou University, Guiyang, PR China. Her recent research interests include federated learning, blockchain, cryptography and information safety. received the M.E degrees in the College of Computer Science and Technology from Guizhou University, China, in 2018 and 2021. Her research interests are cryptography, blockchain, information security and artificial intelligence security.



Keli Liu received the B.S. degree in School of Software from Henan Finance University, Zhengzhou, China, in 2022. She is currently pursuing the M.S. degree with the State Key Laboratory of Public Big Data, Guizhou University, Guiyang, China. Her research interests include edge computing, resource scheduling.



Jing Yang (Member, IEEE) received the PhD degree in mechanical and electronic engineering from Guizhou University, in 2020. From August 2018 to September 2019, he was awarded a scholarship by the China Scholarship Council (CSC) under the State Scholarship Fund to pursue his study with Oklahoma State University, as a Joint Ph.D. student with the school of Computer Science and Technology, where he joined the Guoliang Fan's Group. From October 2022 to October 2025, he was a visiting scholar studying in the team of Wu Fan's (NSFC Excellent

Young Scholars) and a postdoctoral in Professor Guo Minyi (IEEE Fellow) from Shanghai Jiao Tong University. He is currently an assistant professor with the State Key Laboratory of Public Big Data, Guizhou University, China. His research interests include high performance computing, task scheduling in various architectures and Open domain visual learning. He has published more than 50 peer-reviewed papers in the related area, including well-archived international journals such as IEEE Transactions on Circuits and Systems for Video Technology, IEEE Transactions on Green Communications and Networking, IEEE Transactions on Intelligent transportation systems, and IEEE Internet of Things Journal. He serves more than 20 prestigious international journals and conferences, such as IEEE TC, IEEE TMC, IEEE TI, IEEE TIM, IEEE TSC, IEEE TSM, IEEE TAI, IEEE TASE, IEEE IOTJ, IEEE TCE, IEEE TMM, IEEE TCSVT, IEEE TNNLS, IEEE TIFS, IEEE TPDS.



Xu Wang received the PhD degree in Condensed Matter Physics from the Department of Physics, University of Oxford, UK, in 2012. He obtained his Bachelor's degree in Microelectronics from the School of Physical Science and Technology, Sichuan University, China, in 2007. His research interests include Quantum Information, New Energy Materials, and Optoelectronic Devices. He has published more than 60 peer-reviewed papers indexed by SCI/EI in high-impact international journals, including Journal of Materials Chemistry A, Light: Science & Applications, Science China Materials, Advanced Optical Materials, Optics Express, Applied Physics Letters, Journal of Applied Physics, and Nano Letters.



Jianhong Cheng received the BS degree from Liaoning Technical University, Fuxin, China, in 2014, the MS degree in software engineering from Central South University, Changsha, China, in 2017, and the PhD degree in computer science and technology from Central South University, Changsha, China, in 2022. Now, he is an academic leader at the Institute of Guizhou Aerospace Measuring and Testing Technology, China. His research interests include machine learning and deep learning.



Yulong Yang received the BS degree in computer science and technology from Hubei Engineering University, Xiaogan, China, in 2012, the MS degree in computer software and theory from Guizhou University, Guiyang, China, in 2015. Now, He is the deputy director at the Institute of Guizhou Aerospace Measuring and Testing Technology, China. His research interests include are cryptography and network security.



Minyi Guo (Fellow, IEEE) received the BSc and ME degrees in computer science from Nanjing University, China, and the PhD degree in computer science from the University of Tsukuba, Japan. Additionally, he is the Department of Computer Science and Engineering director of the Embedded and Pervasive Computing Center, Shanghai Jiao Tong University. Before joining SJTU, he had been a professor with the School of Computer Science and Engineering, University of Aizu, Japan. He received the national science fund for distinguished young scholars from

NSFC in 2007. His present research interests include parallel/distributed computing, compiler optimizations, Big Data, and cloud computing. He has more than 400 publications in major journals and international conferences in these areas. He received 7 best/highlight paper awards from international conferences including ALSPOS 2017 and ICCD 2018. He is now editor-in-chief of IEEE Transactions on Sustainable Computing, and on the editorial board of IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Cloud Computing, and Journal of Parallel and Distributed Computing. He is the Zhiyuan chair professor, and an ACM distinguished member.